

Glosario - Semana 06

Términos clave de **Relaciones SQLAlchemy** y **Service Layer**.

A

Association Table

Tabla asociativa que conecta dos entidades en una relación N:M. Solo contiene las foreign keys de ambas tablas, sin datos adicionales.

```
post_tags = Table(
    "post_tags",
    Base.metadata,
    Column("post_id", ForeignKey("posts.id"), primary_key=True),
    Column("tag_id", ForeignKey("tags.id"), primary_key=True)
)
```

B

back_populates

Parámetro de `relationship()` que **conecta bidireccionalmente** dos relaciones. Ambos lados deben declararlo apuntando al otro.

```
# En Author
posts = relationship("Post", back_populates="author")

# En Post
author = relationship("Author", back_populates="posts")
```

Business Logic

Lógica de negocio: las reglas y operaciones específicas del dominio de la aplicación. Ejemplos: validar que un email sea único, calcular descuentos, verificar permisos.

C

Cascade

Comportamiento en cascada que define qué sucede con los objetos relacionados cuando se modifica o elimina el objeto padre.

```
posts = relationship("Post", cascade="all, delete-orphan")
# Si se elimina el autor, se eliminan sus posts
```

Opciones comunes:

- `save-update` : Propagar `add()` a relacionados
 - `delete` : Eliminar relacionados al eliminar padre
 - `delete-orphan` : Eliminar si quedan huérfanos
-

D

Dependency Injection (DI)

Inyección de dependencias: patrón donde las dependencias se pasan desde fuera en lugar de crearlas internamente. FastAPI lo implementa con `Depends()`.

```
def get_author(db: Session = Depends(get_db)):
    service = AuthorService(db) # db inyectada
    return service.list_all()
```

E

Eager Loading

Carga anticipada: estrategia que carga las relaciones en la misma consulta o inmediatamente después, evitando el problema N+1.

```
# Carga autor junto con el post
stmt = select(Post).options(joinedload(Post.author))
```

F

ForeignKey

Clave foránea: columna que referencia la primary key de otra tabla. Establece la relación a nivel de base de datos.

```
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))
```

J

joinedload

Estrategia de eager loading que usa **JOIN** para cargar relaciones en una sola query. Óptimo para relaciones 1:1 o N:1.

```
stmt = select(Post).options(joinedload(Post.author))
# SELECT posts.*, authors.* FROM posts JOIN authors ON ...
```

L

Lazy Loading

Carga perezosa: estrategia por defecto donde las relaciones se cargan solo cuando se acceden. Puede causar el problema N+1.

```
post = db.get(Post, 1)
# Query 1: SELECT * FROM posts WHERE id = 1

print(post.author.name)
# Query 2: SELECT * FROM authors WHERE id = ? (lazy load)
```

M

Many-to-Many (N:M)

Relación muchos a muchos: una entidad puede relacionarse con múltiples instancias de otra y viceversa. Requiere tabla asociativa.

```
Post ↔ post_tags ↔ Tag
Un post tiene muchos tags
Un tag está en muchos posts
```

Mapped

Tipo genérico de SQLAlchemy 2.0 que indica una columna mapeada con su tipo Python.

```
id: Mapped[int] = mapped_column(primary_key=True)
name: Mapped[str] = mapped_column(String(100))
bio: Mapped[str | None] = mapped_column(Text, nullable=True)
```

N

N+1 Problem

Problema N+1: antipatrón donde se ejecuta 1 query inicial + N queries adicionales (una por cada relación). Se soluciona con eager loading.

```
# ❌ N+1 Problem
authors = db.query(Author).all() # Query 1
for author in authors:
    print(author.posts) # Query 2, 3, 4... (N queries)

# ✅ Solución con eager loading
authors = db.query(Author).options(selectinload(Author.posts)).all()
# Solo 2 queries total
```

O

One-to-Many (1:N)

Relación uno a muchos: una entidad se relaciona con múltiples instancias de otra. La FK va en el lado "muchos".

```
Author (1) → Posts (N)
Un autor tiene muchos posts
Un post pertenece a un autor
```

ORM (Object-Relational Mapping)

Mapeo objeto-relacional: técnica que convierte datos entre sistemas de tipos incompatibles (objetos Python ↔ tablas SQL).

R

relationship()

Función de SQLAlchemy que define la **relación ORM** entre modelos. No crea columnas en la base de datos.

```
# Define cómo navegar entre objetos
posts: Mapped[list["Post"]] = relationship(back_populates="author")
```

Repository Pattern

Patrón repositorio: abstrae el acceso a datos detrás de una interfaz. Similar a Service Layer pero enfocado solo en persistencia.

```
class AuthorRepository:
    def find_by_id(self, id: int) -> Author | None: ...
    def save(self, author: Author) -> Author: ...
```

S

secondary

Parámetro de `relationship()` que especifica la **tabla asociativa** en relaciones N:M.

```
tags = relationship("Tag", secondary=post_tags, back_populates="posts")
```

selectinload

Estrategia de eager loading que usa **query separada con IN**. Óptimo para relaciones 1:N o N:M.

```
stmt = select(Author).options(selectinload(Author.posts))
# Query 1: SELECT * FROM authors
# Query 2: SELECT * FROM posts WHERE author_id IN (1, 2, 3...)
```

Service Layer

Capa de servicio: patrón arquitectónico que encapsula la lógica de negocio en clases dedicadas, separándola de los endpoints HTTP.

```
Router (HTTP) → Service (Lógica) → Model (Datos)
```

Session

Objeto SQLAlchemy que gestiona la **unidad de trabajo**: tracking de cambios, transacciones, y comunicación con la base de datos.

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

U

unique()

Método de SQLAlchemy 2.0 necesario después de queries con `joinedload()` para **eliminar duplicados** en colecciones.

```
stmt = select(Post).options(joinedload(Post.tags))
posts = db.execute(stmt).scalars().unique().all()
```

Comparativa: Estrategias de Loading

Estrategia	Queries	Mejor para	Ejemplo
Lazy (default)	1 + N	Relaciones poco usadas	Acceso esporádico
joinedload	1 (JOIN)	N:1, 1:1	Post → Author
selectinload	2 (IN)	1:N, N:M	Author → Posts
subqueryload	2 (subquery)	1:N complejas	Alternativa a selectin

Comparativa: Service vs Repository

Aspecto	Service Layer	Repository Pattern
Responsabilidad	Lógica de negocio	Acceso a datos
Conoce reglas	✔ Sí	✗ No
Transacciones	Coordina	No maneja
Dependencias	Puede usar repositories	Solo DB

 **Tip:** Estos términos aparecerán constantemente en desarrollo backend. Asegúrate de entenderlos bien antes de avanzar a la siguiente semana.